Choose Your Fate

Exam Project Description

Software Quality - RPG Choose Your Fate

Projekt	Choose Your Fate
Fag	Software Quality - Exam Project
Type	Webbaseret text-RPG med frontend, backend, database og ekstern API
Gruppe	Sebastian Juul Knudsen, Patrick Fabrin, Mikkel Olsen og Alexander Doctor Heyde Rasmussen
Dato	19. maj 2026

Project Overview
Choose Your Fate is a browser-based, text-driven role-playing game inspired by classic choice-driven adventure books. The system combines a React frontend, a Spring Boot REST API, a MySQL database layer, JWT-based authentication, an ElevenLabs text-to-speech integration, and a primary/secondary database setup for availability and failover.

1. Review

One group member authors the Software Requirements Specification(SRS) for Choose Your Fate. The SRS describes the current vertical slice: account/login, character creation, scene navigation, choices, persistence, external API usage, and database availability.

- The group members(including author) conduct a formal review of the SRS.
- The review focuses on completeness, correctness, clarity, consistency, testability, and alignment with the course requirements.
- The current review log contains 37 findings with severity, responsible person, recommendation, and status.

Deliverables

Deliverable	Format	Indhold
Software Requirements Specification	PDF	Functional requirements, non-functional requirements, technology requirements, user stories, and acceptance-oriented slice definition.
SRS Review Report	PDF	Formal review overview, roles, logistics, severity scale, defect log, follow-up actions, and sign-off.

2. Risk Analysis

The project uses one consolidated risk analysis. The analysis groups risks into technical risks and general project risks, and each risk is tracked with probability, impact, risk factor, current risk factor, mitigation, responsible person, date, follow-up, and status.

- Probability and impact are both rated on a 1-5 scale. The risk factor is calculated from those values, and the current RF shows the updated state after mitigation or follow-up work.
- The technical risks include authorization, third-party API tokens/shutdown, code loss, false failover, backup freshness, transaction consistency, and proving failover through realistic testing.
- The general risks include sickness, team member absence, interpersonal problems, funding, documentation quality, hardware failure, and merge/push policy. The highest-priority open risks are made visible in one shared risk matrix instead of being split across phases.

Deliverables collected in one PDF

Deliverable	Format	Indhold
Consolidated risk analysis	PDF	One shared risk document containing technical risks, general risks, probability, impact, RF/current RF, mitigation, owner, follow-up, and status.
Risk matrix	PDF	One current likelihood-vs-impact matrix showing the combined state of the project risks.

3. Black-Box Test Design

Black-box techniques are applied to the public behaviour of the application without relying on internal implementation details. Test cases are derived from the SRS, API contract, validation rules, roles, and gameplay flow.

- Equivalence partitioning is used for username/password input, JWT-authenticated vs unauthenticated requests, role access, valid/invalid character IDs, and valid/invalid entity relationships.
- Boundary value analysis is used for text lengths, required fields, numerical character details, response time expectations, and edge cases such as missing or null values.
- State transition testing is relevant for scene progression, character path updates, and database failover/failback states.
- Decision tables are relevant for authorization behaviour: ADMIN access, USER ownership access, unauthorized access, and invalid role scenarios.

Deliverables collected in one PDF

Deliverable	Format	Indhold
Equivalence partitioning analysis	PDF	Input classes and resulting positive/negative test cases for authentication, characters, details, path, equipment, race details, and story data.
Boundary value analysis	PDF	Boundary-focused test cases for validation, IDs, required fields, response time, and

		request payloads.
Decision tables and state transitions	PDF	Authorization decision table and state model for gameplay and availability transitions where relevant.

4. Static Testing Tools and White-Box Test Design

Static tools and white-box analysis are used to inspect internal code quality and guide additional tests. The project is structured around a Spring Boot backend, a React/TypeScript frontend, and SQL scripts for schema, seed data, functions, procedures, triggers, events, and security roles.

- SonarQube is used for static analysis of bugs, vulnerabilities, maintainability issues, duplication, and code smells.
- Backend coverage is measured using IntelliJ IDEA's built-in test coverage tool by running the backend test suite with coverage enabled.
- Frontend linting is available through ESLint and TypeScript checks.
- White-box testing focuses on service logic, authorization checks, repository interactions, failover state transitions, replication queue behaviour, and exception paths.

Deliverables as one PDF

Deliverable	Format	Indhold
Static analysis report	PDF	Screenshots or exported findings from SonarQube/linters with short explanation of the most important issues.
Coverage report	PDF	Coverage overview and explanation of how low-coverage paths were used to design more unit tests.

5. Unit Testing and Integration Testing

The backend contains automated JUnit tests under src/test/java. The implemented JUnit tests focus on availability logic, including database routing decisions, failover/failback state transitions, primary health evaluation, handling of queued database synchronization jobs, and write operation coordination. These tests are primarily unit-level tests because they verify isolated service behaviour and state transitions. API endpoint tests and end-to-end UI tests are implemented with Playwright.

- Unit tests verify business logic and state transitions without requiring the full application flow.
- Integration tests verify database-backed behaviour and Spring wiring where direct unit tests are not enough.
- Black-box test cases from the design phase is represented in automated unit or integration tests, with parameterised tests/data providers where applicable.

Deliverables

Deliverable	Format	Indhold
Application source code	Repository folder	Spring Boot backend, React frontend, MySQL scripts, and Docker Compose setup.
Unit and integration test source	Repository folder	JUnit test classes under src/test/java, with focus on assertions for both success and failure paths.

6. Continuous Testing

Continuous testing is implemented in the github repository using a CI pipeline triggered on push and pull request to main. The pipeline must verify potential integration errors before code is merged.

- Backend pipeline: builds the Spring Boot application and runs the automated JUnit tests.
- Frontend pipeline: installs dependencies, runs the TypeScript build, runs ESLint, and executes Playwright tests.
- Performance CI: k6 load/stress/spike tests, is run manually or on a timed schedule.

Deliverables

Deliverable	Format	Indhold
CI pipeline script	YAML	Workflow file that builds backend/frontend and runs automated tests.
CI output evidence	PDF	Pipeline run output or screenshots showing build and test results.

7. API Testing

API testing is implemented as coded Playwright tests. The tests cover internal REST endpoints for both admin and normal user flows, including authentication, authorization, ownership rules, positive requests, negative requests, and expected HTTP/JSON responses.

- Playwright API tests verify HTTP status codes.
- Tests verify JSON fields, response bodies, error responses, authorization behaviour, and response time expectations.
- Positive tests include valid login, character creation, retrieving own data, updating character details, character path, equipment, and reading public race details.
- Negative tests include missing JWT, wrong role, accessing another user's data, invalid references, and deleting protected data.
- Endpoint coverage includes authentication, accounts, characters, character details, character paths, chapters, scenes, choices, inventory, equipment, items, quests, NPCs, race details, AI, TTS, and availability.

Deliverables

Deliverable	Format	Indhold
API test collection	JSON	Playwright API test source code with request setup, assertions, and reusable test data.
API environment	JSON	Playwright configuration and test setup for base URL, authentication flow, JWT handling, and seeded admin/user test data.

8. End-to-End UI Testing

End-to-end UI testing is implemented in code using Playwright. The tests validate the user-facing behaviour across the React frontend, Spring Boot backend, authentication flow, and persisted game state.

- Primary journeys include register/login, account access, character creation, opening the game, displaying a scene, selecting a choice, and verifying story progression.
- Authentication and authorization journeys verify that protected pages require login and that users only access their own relevant data.
- Validation journeys cover invalid login, missing required fields, invalid character creation, and visible error feedback.
- Gameplay regression journeys cover inventory/equipment display and character path updates where they are part of the implemented slice.

Deliverables

Deliverable	Format	Indhold
E2E test source code	Repository folder	Playwright E2E UI test source code with browser flows, assertions, and test setup.

9. Stress and Performance Testing

Load, stress, and spike testing are implemented as code-based k6 tests against the backend API. The k6 scripts define traffic scenarios, thresholds, checks, and metrics so the performance tests can be repeated consistently and be part of our CI.

- Load tests simulate expected traffic across key API flows such as login, character retrieval, scene retrieval, and choice submission.
- Stress tests gradually increase virtual users to identify when response time, throughput, or error rate becomes unacceptable.
- Spike tests introduce sudden bursts of traffic to evaluate how the API behaves under unexpected load.

Deliverables

Deliverable	Format	Indhold
Performance test reports	PDF/screenshots	Evidence for load, stress, and spike scenarios, including metrics and short interpretation.

Delivery Structure

The final hand-in will be delivered as one zip file with one folder per task. The structure below mirrors the exam assignment and makes it clear where each deliverable belongs.

Deliverable	Format	Content
01-review	Folder	SRS PDF and SRS review PDF.
02-risk-analysis	Folder	Consolidated risk analysis and one current risk matrix.
03-black-box-test-design	Folder	Equivalence partitioning, boundary value analysis, decision/state tests where relevant.
04-static-white-box	Folder	Static analysis evidence, coverage report, and white-box test design notes.
05-unit-integration-tests	Folder/source	Application source and automated unit/integration tests.
06-continuous-testing	Folder	CI YAML and CI output evidence.
07-api-testing	Folder	API collection and environment JSON.
08-e2e-ui-testing	Folder/source	Playwright or equivalent E2E test source code.
09-performance-testing	Folder	Load, stress, and spike test reports/screenshots.
10 – Full source code for the project	Source folder	Full project source code, including frontend, backend, database scripts, tests, CI workflows, and supporting project files.